Глубокое обучение с подкреплением

Модуль 3: глубокое Q-обучение

Курс Deep RL от Hugging Face

Полное учебное издание на основе исходных MDX-файлов

Исходный материал: курс Deep RL от Hugging Face.

Содержание

1 Глубокое Q-обучение **2**

2 От Q-обучения к глубокому Q-обучению **3**

3 Глубокая Q-сеть (DQN) **5**

3.1 Предобработка входа и проблема временной информации 5

4 Алгоритм глубокого Q-обучения **7**

4.1 Воспроизведение опыта для более эффективного обучения 9

4.2 Фиксированная целевая Q-сеть для стабилизации обучения 11

4.3 Двойная DQN (Double DQN) 14

5 Практическое занятие **15**

5.1 Модуль 3: Deep Q-Learning в играх Atari (Space Invaders) с помощью RL Baselines3 Zoo 16

5.2 Обучим агента Deep Q-Learning играть в Atari Space Invaders и загрузим его в Hub 17

5.3 Установка RL-Baselines3 Zoo и зависимостей (материал для чтения) 18

6 Проверка знаний **24**

6.1 Ответы на вопросы и пояснения 24

7 Заключение **26**

8 Глоссарий **27**

9 Дополнительная литература **28**

1 Глубокое Q-обучение

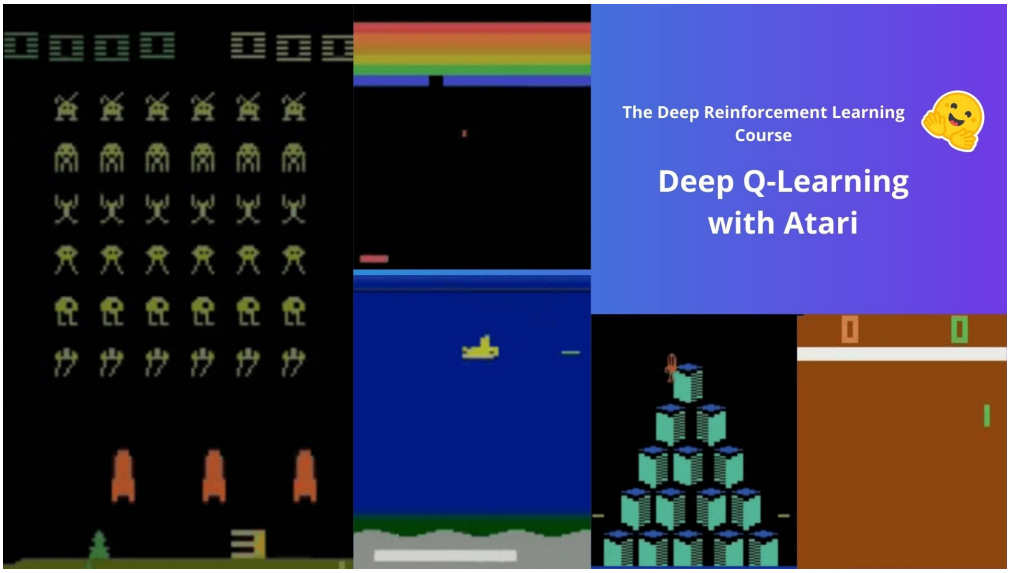


Рис. 1: Обложка модуля 3: глубокое Q-обучение [оригинал]

В предыдущем модуле мы познакомились с первым алгоритмом обучения с подкреплением — Q-обучением, **реализовали его с нуля** и обучили агента в двух средах: FrozenLake-v1 (замёрзшее озеро) и Taxi-v3 (такси).

С помощью этого простого алгоритма мы получили отличные результаты. Однако обе среды были относительно простыми, поскольку **пространство состояний в них было небольшим и дискретным**: 16 состояний в FrozenLake-v1 и 500 в Taxi-v3. Для сравнения, пространство состояний в играх Atari может **содержать от** 10^9 **до** 10^{11} **состояний**.

Но, как мы увидим дальше, **создавать и обновлять Q-таблицу в средах с большим пространством состояний становится неэффективно**.

Поэтому в этом модуле мы **изучим первого агента глубокого обучения с подкреплением** — Deep Q-Learning. Вместо Q-таблицы он использует нейронную сеть, которая получает состояние на вход и аппроксимирует Q-значения всех возможных действий для этого состояния.

Кроме того, **мы обучим агента играть в Space Invaders и другие игры Atari с помощью RL-Zoo**. Это инструментарий обучения с подкреплением на базе Stable-Baselines, содержащий скрипты для обучения и оценки агентов, подбора гиперпараметров, построения графиков и записи видео.

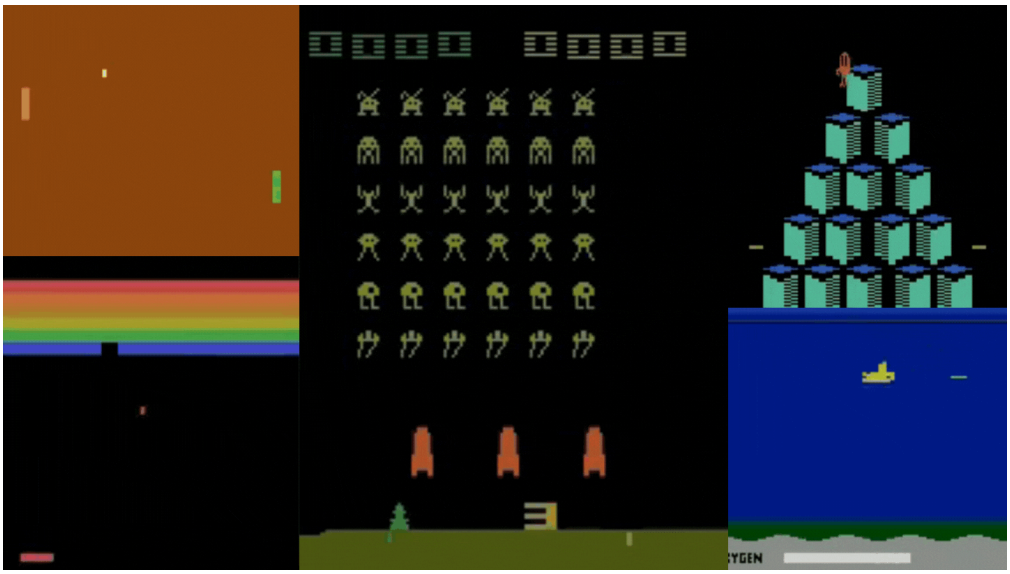


Рис. 2: Примеры сред Atari, в которых можно обучать агентов [оригинал]

Приступим!

2 От Q-обучения к глубокому Q-обучению

Мы узнали, что **Q-обучение — это алгоритм для обучения Q-функции**, то есть **функции ценности действия (action-value function)**, которая определяет ценность выбора конкретного действия в конкретном состоянии.

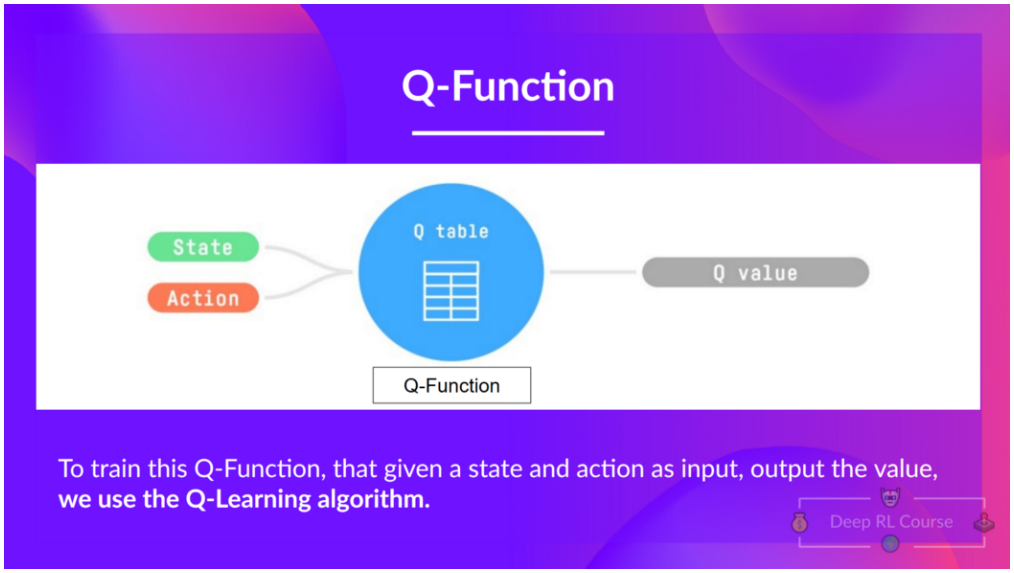


Рис. 3: Табличная Q-функция: значения для пар «состояние–действие» [оригинал]

Название **Q происходит от слова «качество» (quality) действия в данном состоянии**.

Внутри нашего алгоритма Q-функция представлена **Q-таблицей, каждая ячейка которой соответствует ценности некоторой пары «состояние–действие»**. Можно представить эту таблицу как **память или шпаргалку Q-функции**.

Проблема заключается в том, что Q-обучение — *табличный метод*. Если пространства состояний и действий **слишком велики, чтобы эффективно представить их массивами и таблицами**, такой метод перестаёт быть практичным. Иными словами, он **плохо масштабируется**. Q-обучение хорошо работало в небольших средах, например:

- В FrozenLake было 16 состояний.

- В Taxi-v3 было 500 состояний.

Но сегодня мы хотим научить агента играть в Space Invaders — более сложную игру, где входными данными служат изображения отдельных кадров.

Как отмечал Никита Мелькозеров, **пространство наблюдений Atari** имеет размерность (210, 160, 3)*. Значения компонентов находятся в диапазоне от 0 до 255; таким образом, всего возможно $256^{210\times160\times3} = 256^{100800}$ разных наблюдений. Для сравнения: число атомов в наблюдаемой Вселенной оценивается примерно в 10^{80}.

- Один кадр Atari — это изображение размером 210 на 160 пикселей. Поскольку оно цветное (RGB), используются три канала; отсюда размерность (210, 160, 3). Значение каждого канала каждого пикселя может изменяться от 0 до 255.

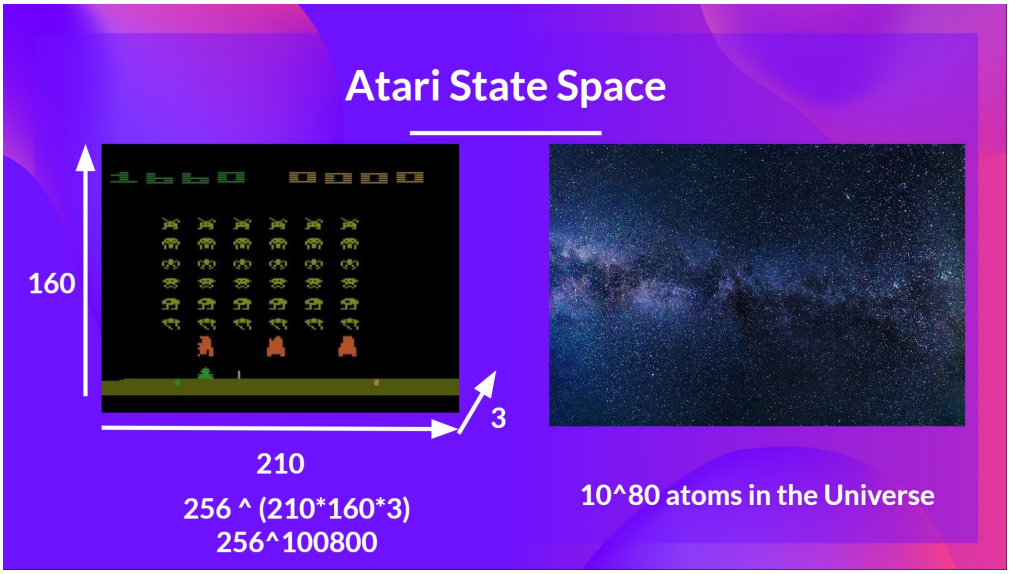


Рис. 4: Огромное пространство наблюдений, представленное изображениями Atari [оригинал]

Следовательно, пространство состояний чрезвычайно велико, и создавать и обновлять Q-таблицу для такой среды неэффективно. Вместо этого разумнее аппроксимировать Q-значения параметризованной Q-функцией $Q_\theta(s, a)$.

Получая состояние на вход, нейронная сеть будет оценивать Q-значения всех допустимых действий в этом состоянии. Именно так и работает глубокое Q-обучение.

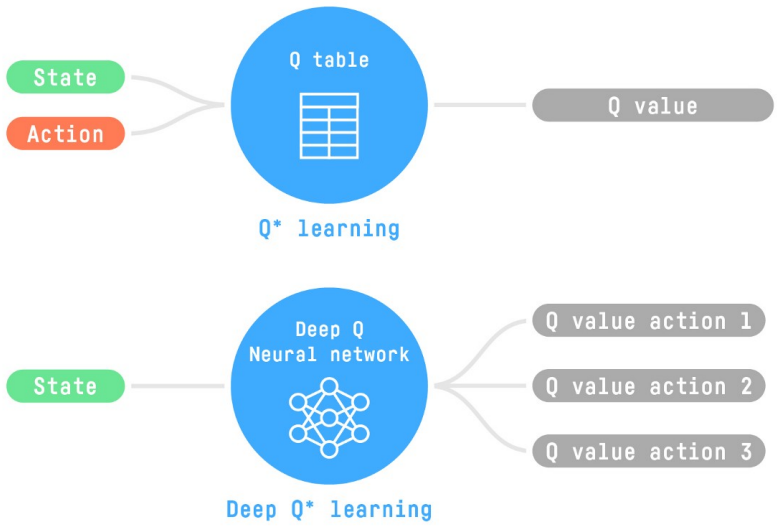


Рис. 5: Нейронная сеть аппроксимирует ценности действий вместо использования Q-таблицы [оригинал]

Теперь, разобравшись в общей идее Deep Q-Learning, рассмотрим подробнее архитектуру Deep Q-Network.

3 Глубокая Q-сеть (DQN)

Вот как устроена нейронная сеть для глубокого Q-обучения:

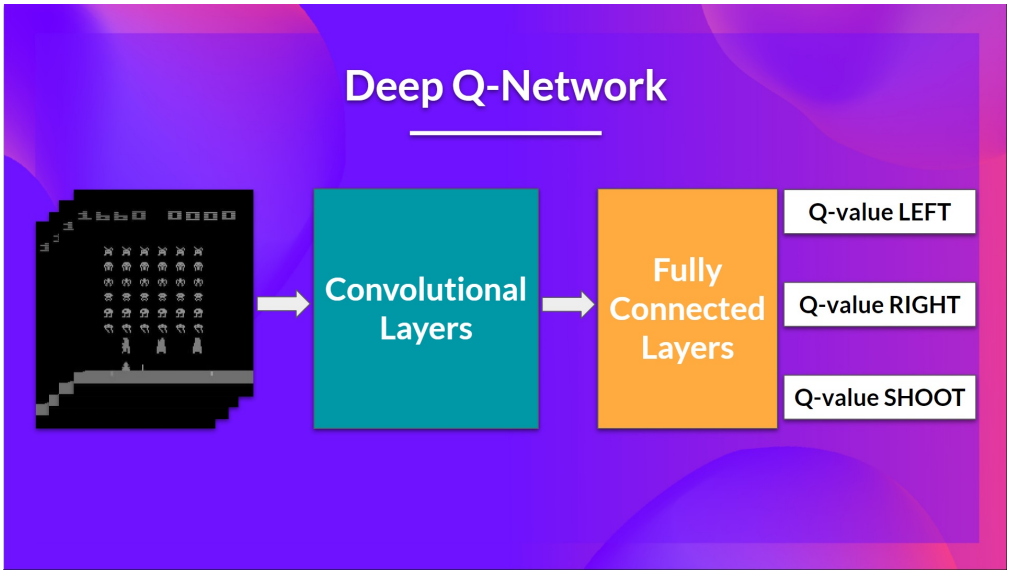


Рис. 6: Архитектура DQN: стек последовательных кадров и оценки ценности действий [оригинал]

На вход сети подаётся **стек из четырёх кадров**, представляющий состояние, а на выходе получается **вектор Q-значений для всех возможных действий в этом состоянии**. Затем, как и в обычном Q-обучении, мы выбираем действие с помощью ε-жадной стратегии.

При инициализации нейронной сети **оценки Q-значений крайне неточны**. Однако в ходе обучения агент DQN начинает связывать игровые ситуации с подходящими действиями и **учится успешно играть**.

3.1 Предобработка входа и проблема временной информации

Необходимо **предобрабатывать входные данные**. Это важный этап: мы стремимся **уменьшить сложность представления состояния и сократить вычислительные затраты на**

обучение.

Для этого мы **уменьшаем размер изображения до 84 на 84 пикселя и переводим его в оттенки серого**. Цвета в играх Atari обычно не несут критически важной информации, поэтому мы можем **заменить три канала RGB одним каналом** и существенно сократить объём входных данных.

В некоторых играх можно также **обрезать неинформативную часть экрана**. После этого мы объединяем четыре последовательных кадра в один стек.

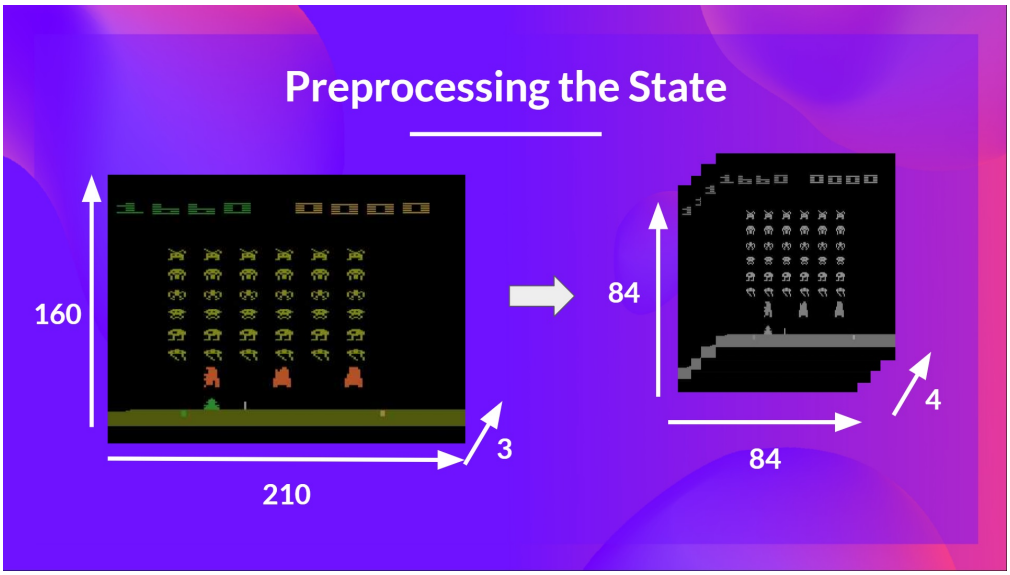


Рис. 7: Предобработка наблюдений Atari и формирование стека кадров [оригинал]

Зачем объединять четыре кадра? Это позволяет **устранить недостаток временной информации**. Рассмотрим пример с игрой Pong. Допустим, мы видим такой кадр:

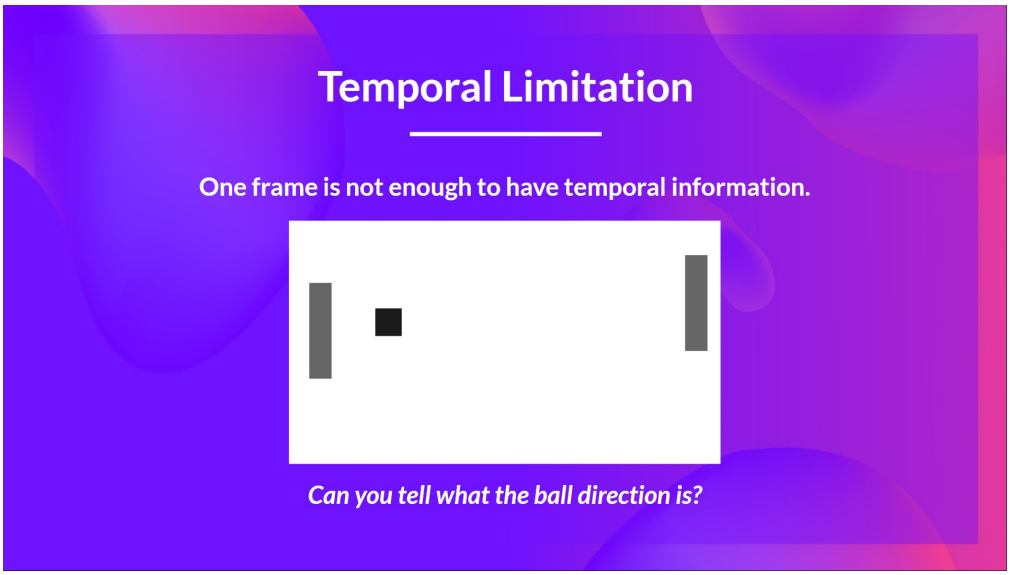


Рис. 8: По одному кадру Pong нельзя определить скорость и направление движения мяча [оригинал]

Можете ли вы сказать, куда движется мяч? Нет: одного кадра недостаточно, чтобы понять движение! А если добавить ещё три кадра? **Теперь видно, что мяч движется вправо**.

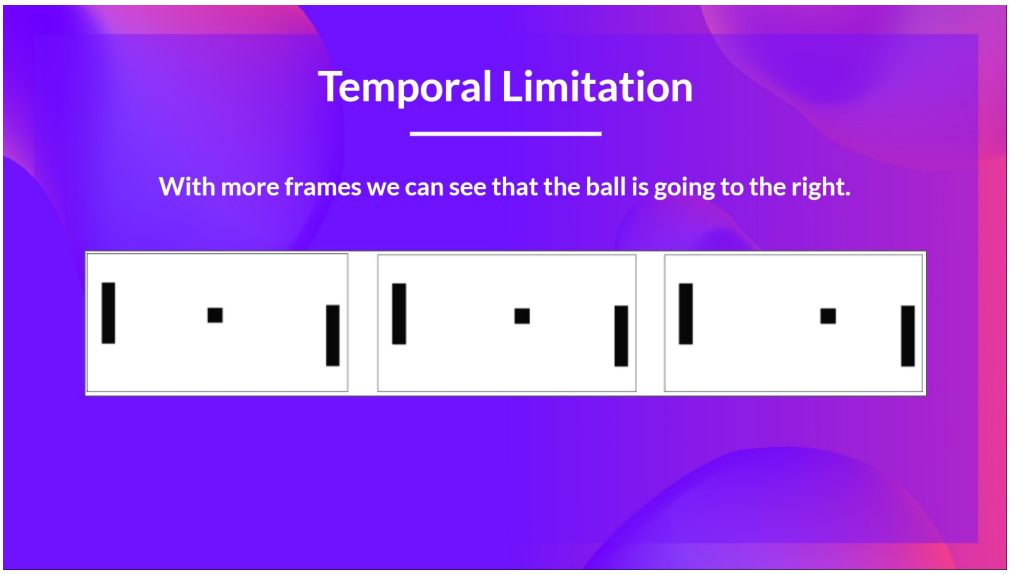


Рис. 9: Несколько последовательных кадров Pong позволяют определить направление мяча [оригинал]

Именно поэтому для получения информации о движении мы объединяем четыре кадра.

Далее стек кадров проходит через три свёрточных слоя. Эти слои **извлекают и используют пространственные зависимости на изображениях**. Поскольку кадры объединены, сеть также **может использовать некоторые временные зависимости между ними**.

Если вы пока не знаете, что такое свёрточные слои, не переживайте. Можно изучить четвёртый урок бесплатного курса по глубокому обучению от Udacity.

Наконец, несколько полносвязных слоёв формируют Q-значения для всех возможных действий в данном состоянии.

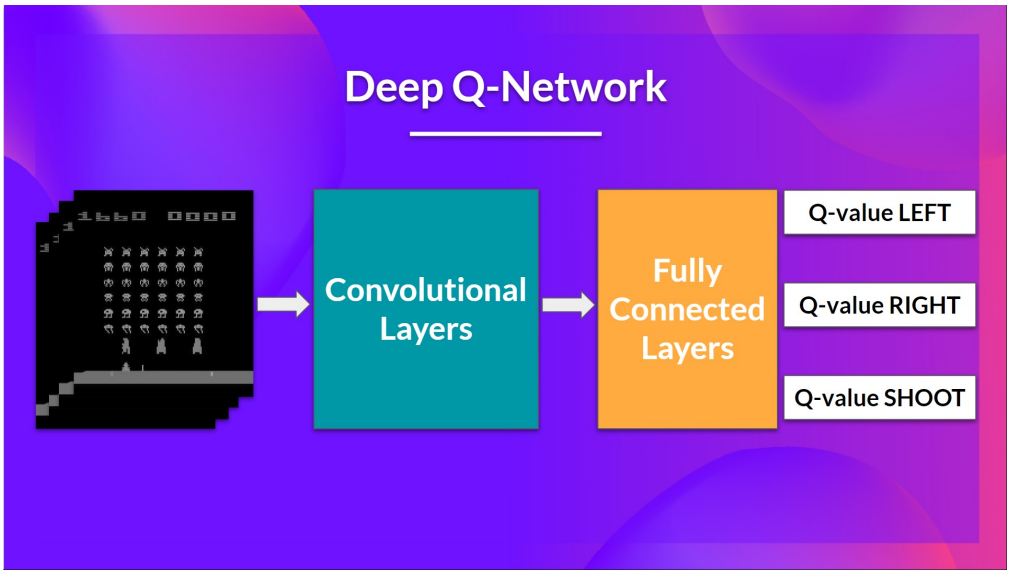


Рис. 10: Архитектура DQN: стек последовательных кадров и оценки ценности действий [оригинал]

Итак, Deep Q-Learning использует нейронную сеть, которая по состоянию аппроксимирует Q-значения всех доступных действий. Теперь изучим сам алгоритм обучения.

4 Алгоритм глубокого Q-обучения

Мы выяснили, что глубокое Q-обучение **использует глубокую нейронную сеть для аппроксимации Q-значений всех возможных действий в каждом состоянии** (оценки функции ценности).

Отличие заключается в том, что во время обучения мы не обновляем Q-значение пары «состояние–действие» непосредственно, как в обычном Q-обучении:

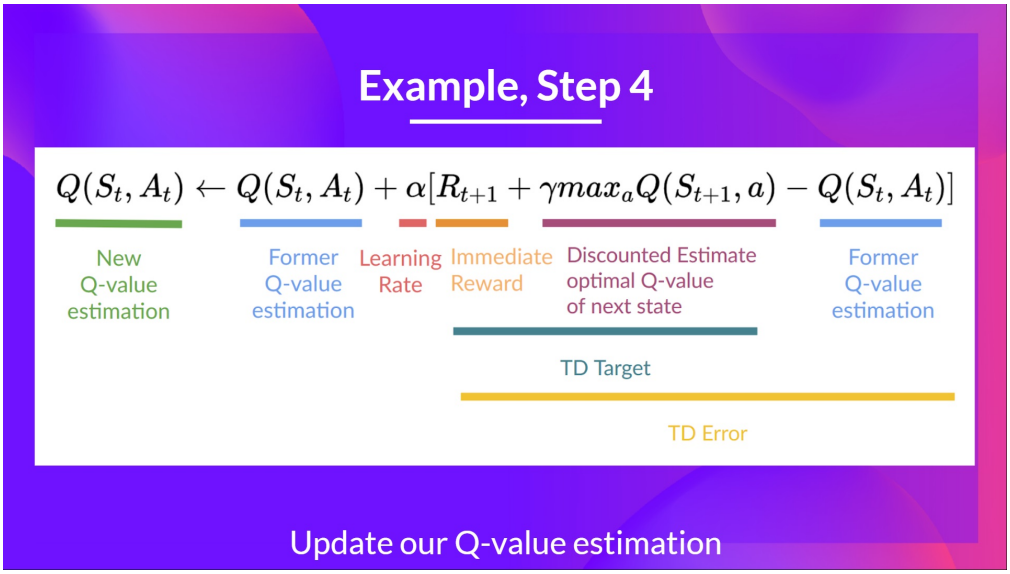


Рис. 11: Обновление в табличном Q-обучении как отправная точка [оригинал]

в Deep Q-Learning мы определяем **функцию потерь, сравнивающую предсказанное Q-значение с целевым Q-значением. Затем с помощью градиентного спуска обновляем веса глубокой Q-сети**, чтобы она лучше аппроксимировала Q-значения.

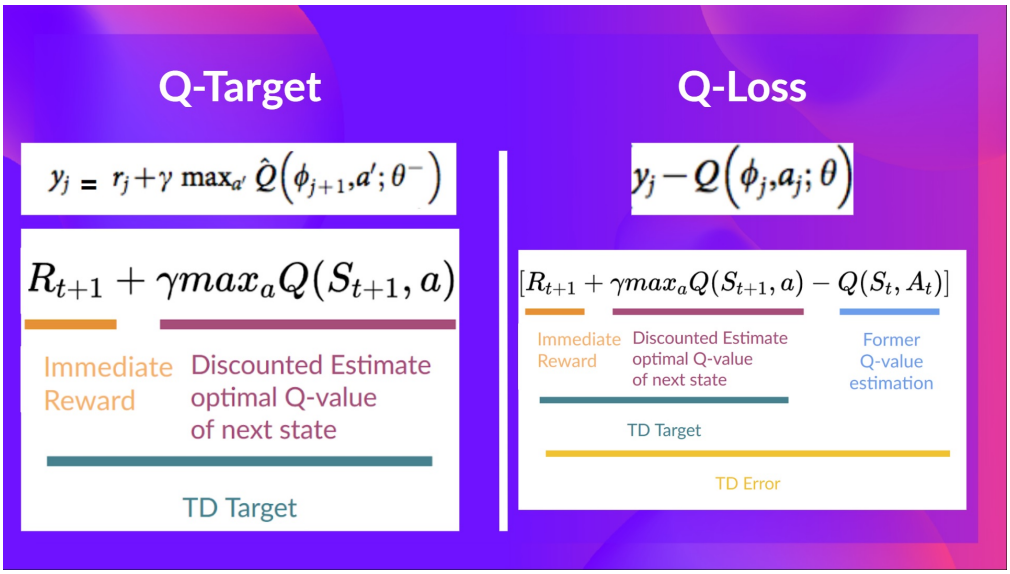


Рис. 12: Функция потерь Deep Q-Learning и целевое значение TD [оригинал]

Алгоритм обучения Deep Q-Learning включает *две фазы*:

- **Сбор опыта (sampling):** агент совершает действия и **сохраняет наблюдаемые переходы в буфере воспроизведения опыта (replay memory)**.
- **Обучение (training):** мы случайным образом выбираем **небольшой пакет переходов и обучаемся на нём, выполняя шаг обновления весов градиентным спуском**.

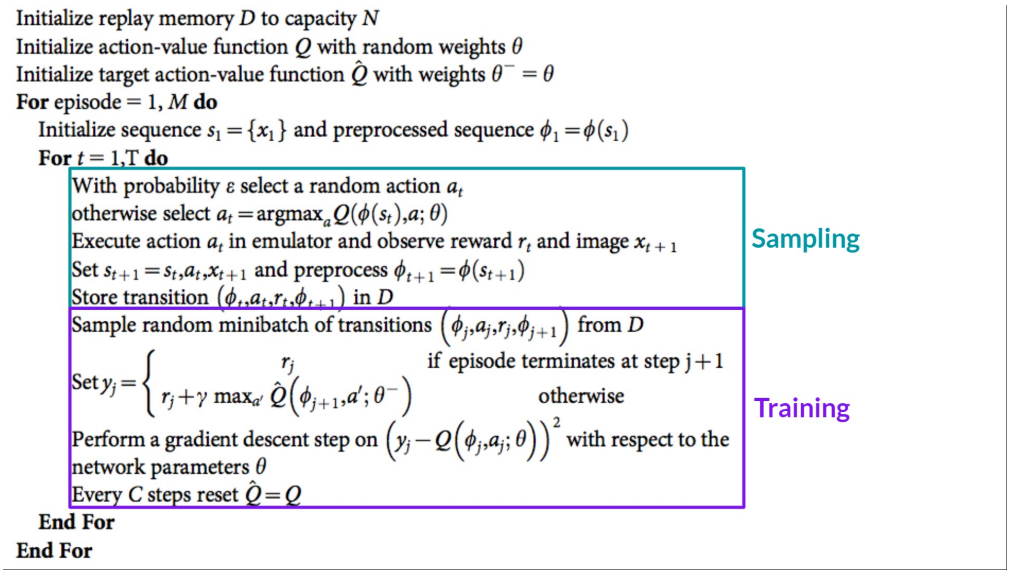


Рис. 13: Две фазы Deep Q-Learning: сбор опыта и обучение [оригинал]

Но это не единственное отличие от табличного Q-обучения. Обучение Deep Q-Learning **может быть нестабильным**, главным образом из-за сочетания нелинейной функции Q-значений (нейронной сети) и бутстрэппинга (когда целевые значения обновляют на основе уже имеющихся оценок, а не фактически полученной полной суммы вознаграждений).

Чтобы стабилизировать обучение, применяют три разных приёма:

1. *Воспроизведение опыта (Experience Replay)* — для **более эффективного использования накопленных переходов**.
2. *Фиксированная целевая Q-сеть (Fixed Q-Target)* — **для стабилизации обучения**.
3. *Двойное глубокое Q-обучение (Double DQN)* — **для уменьшения переоценки Q-значений**.

Рассмотрим каждый из них!

4.1 Воспроизведение опыта для более эффективного обучения

Зачем нам нужна память воспроизведения опыта?

В Deep Q-Learning механизм Experience Replay выполняет две функции:

1. **Более эффективное использование опыта во время обучения**. При обычном онлайн-обучении с подкреплением агент взаимодействует со средой, получает переходы (состояние, действие, вознаграждение и следующее состояние), учится на них (обновляет нейронную сеть), а затем отбрасывает. Это неэффективно.

Experience Replay позволяет **эффективнее использовать уже полученные данные**. Мы храним переходы в буфере воспроизведения и **повторно используем их на протяжении обучения**.

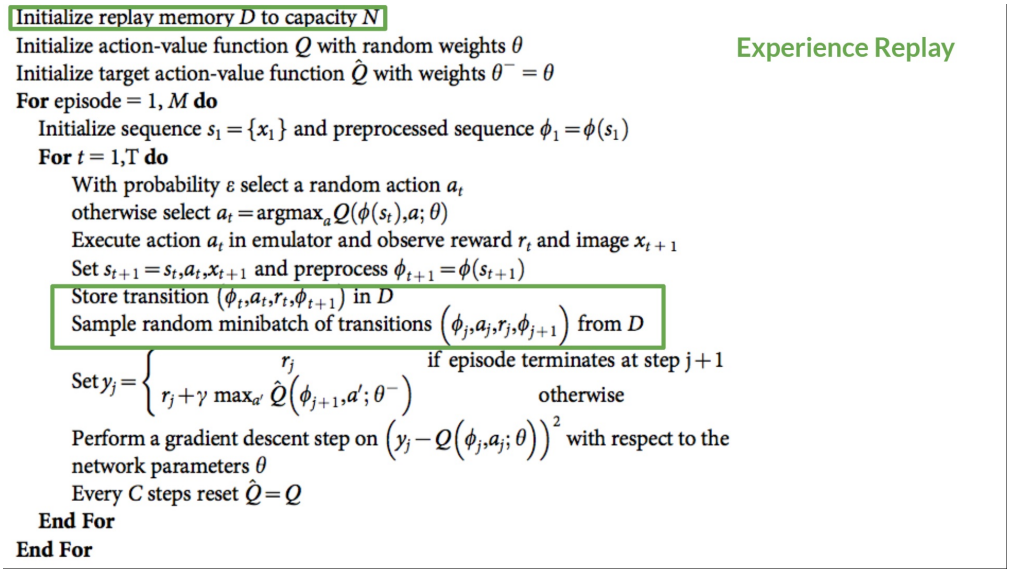


Рис. 14: Буфер воспроизведения опыта, сохраняющий предыдущие переходы [оригинал]

=> Благодаря этому агент может **обучаться на одних и тех же переходах многократно**.

2. **Предотвращение забывания предыдущего опыта (катастрофической интерференции) и уменьшение корреляции между последовательными переходами**.

- **Катастрофическое забывание**: если подавать нейронной сети последовательные переходы, она склонна **забывать предыдущий опыт по мере поступления нового**. Например, когда агент переходит с первого уровня игры на сильно отличающийся второй, он может забыть, как успешно проходить первый уровень.

Решение — сохранять переходы при взаимодействии со средой в буфере Replay Buffer, а затем выбирать оттуда небольшие случайные пакеты. Это не позволяет **нейросети учиться исключительно на том, что агент делал непосредственно перед обновлением**.

У воспроизведения опыта есть и другие преимущества. Случайная выборка уменьшает корреляции между последовательными наблюдениями и помогает предотвратить **сильные колебания или расходимость оценок ценности действий**.

В псевдокоде Deep Q-Learning мы **создаём буфер памяти D вместимостью N** (N — задаваемый гиперпараметр). Затем сохраняем в нём переходы и во время обучения выбираем из буфера пакет опыта для передачи в глубокую Q-сеть.

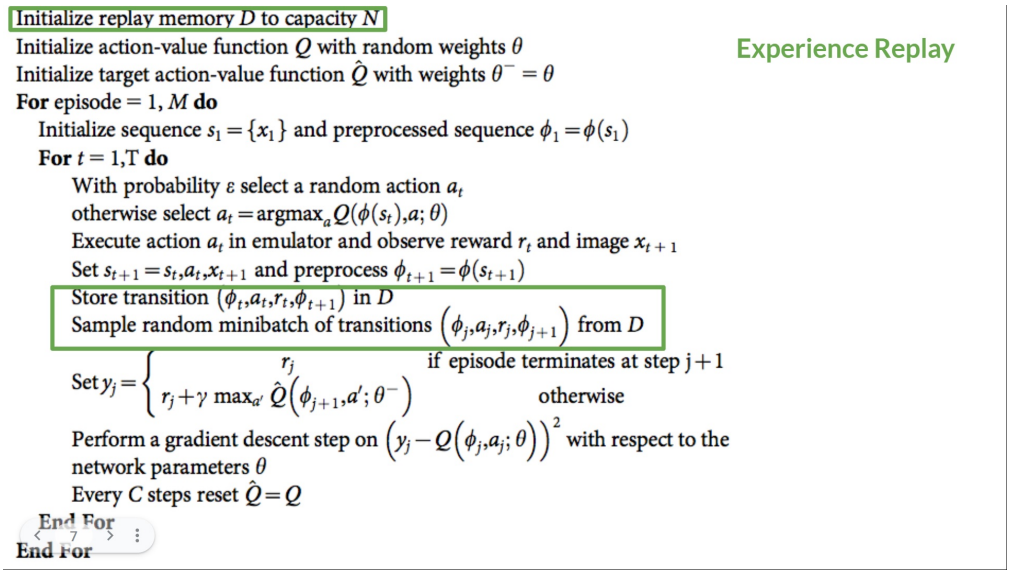


Рис. 15: Псевдокод DQN: использование буфера воспроизведения опыта [оригинал]

4.2 Фиксированная целевая Q-сеть для стабилизации обучения

При вычислении ошибки временных разностей (TD error, то есть функции потерь) мы находим **разность между целевым TD-значением (Q-Target) и текущей оценкой Q-значения**.

Однако **истинное целевое TD-значение нам неизвестно**, и его приходится оценивать. Из уравнения Беллмана следует, что цель равна вознаграждению за действие в текущем состоянии плюс дисконтированное максимальное Q-значение в следующем состоянии.

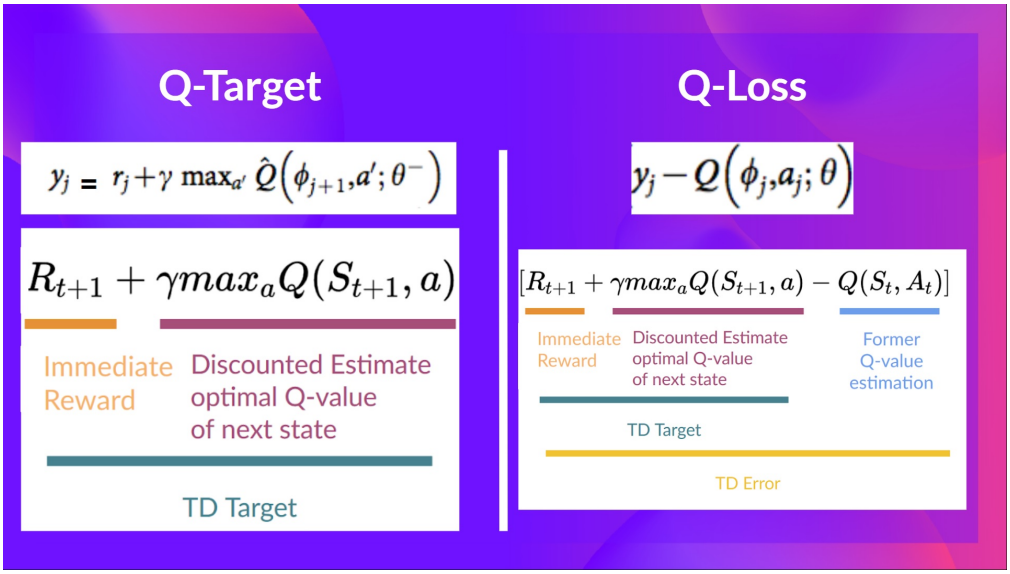


Рис. 16: Функция потерь Deep Q-Learning и целевое значение TD [оригинал]

Проблема в том, что для вычисления целевого TD-значения **и** текущей оценки Q мы используем одни и те же параметры (веса). Поэтому между целевым значением и изменяемыми параметрами возникает сильная взаимосвязь.

Следовательно, на каждом шаге обучения **смещаются и наши Q-оценки, и целевые значения**. Мы приближаемся к цели, но она сама постоянно движется. Это похоже на погоню за движущейся мишенью и может вызывать значительные колебания процесса обучения.

Представьте, что вы ковбой (текущая Q-оценка) и хотите поймать корову (целевое Q-значение). Ваша задача — приблизиться к ней (уменьшить ошибку).

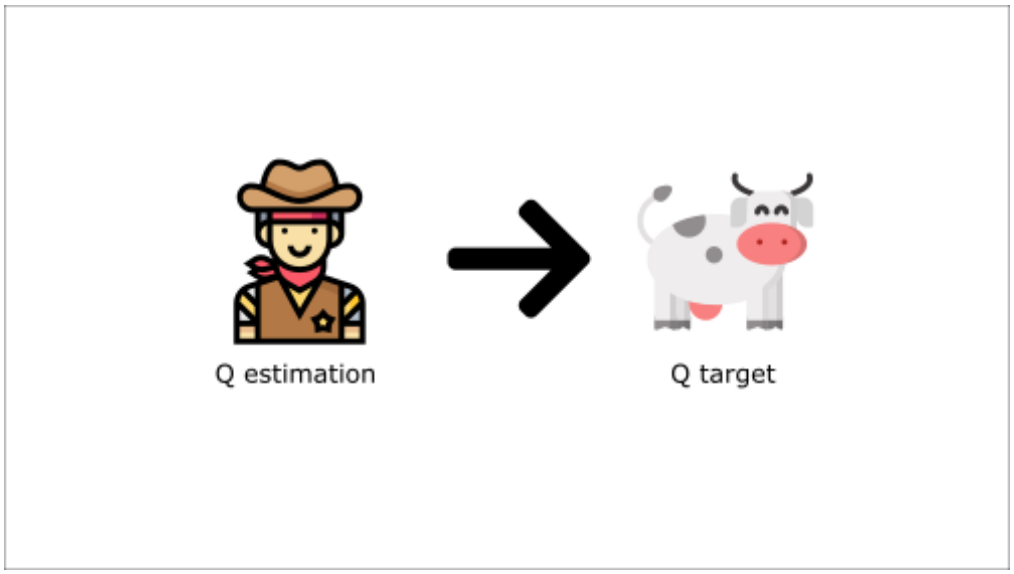


Рис. 17: Аналогия с движущейся целью: первый шаг обучения Q-функции [оригинал]

На каждом временном шаге вы пытаетесь приблизиться к корове, но она тоже перемещается на каждом шаге (потому что используются одни и те же параметры).

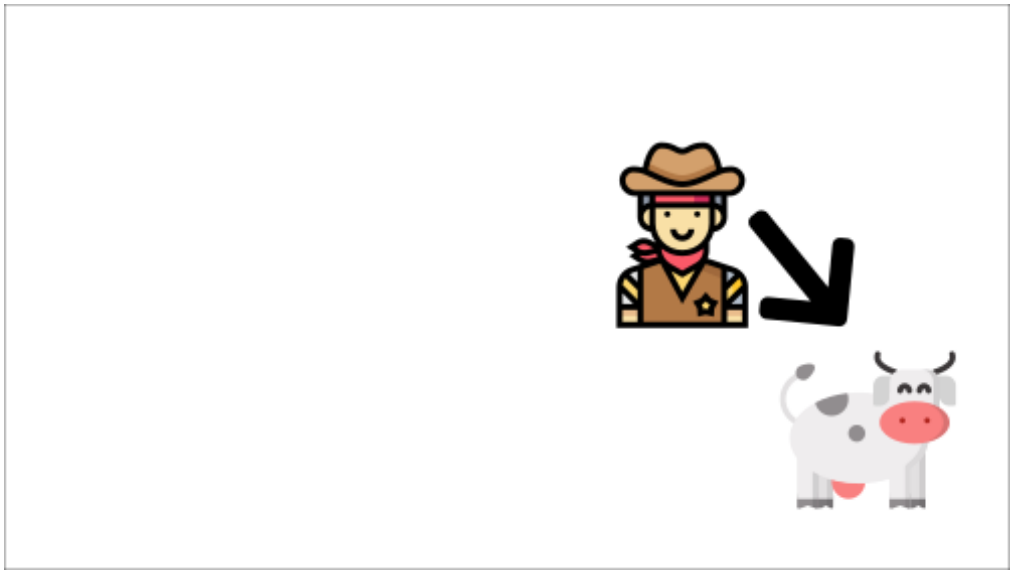


Рис. 18: Движущаяся цель: смещение целевого значения [оригинал]

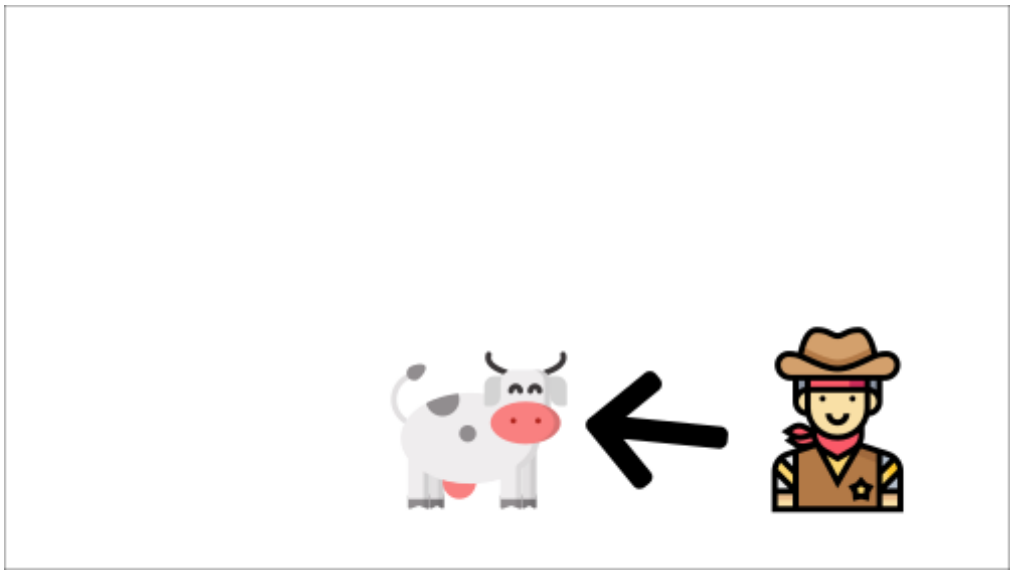


Рис. 19: Движущаяся цель: последовательные изменения положения цели [оригинал]

В результате траектория преследования становится странной и извилистой, что соответствует сильным колебаниям во время обучения.

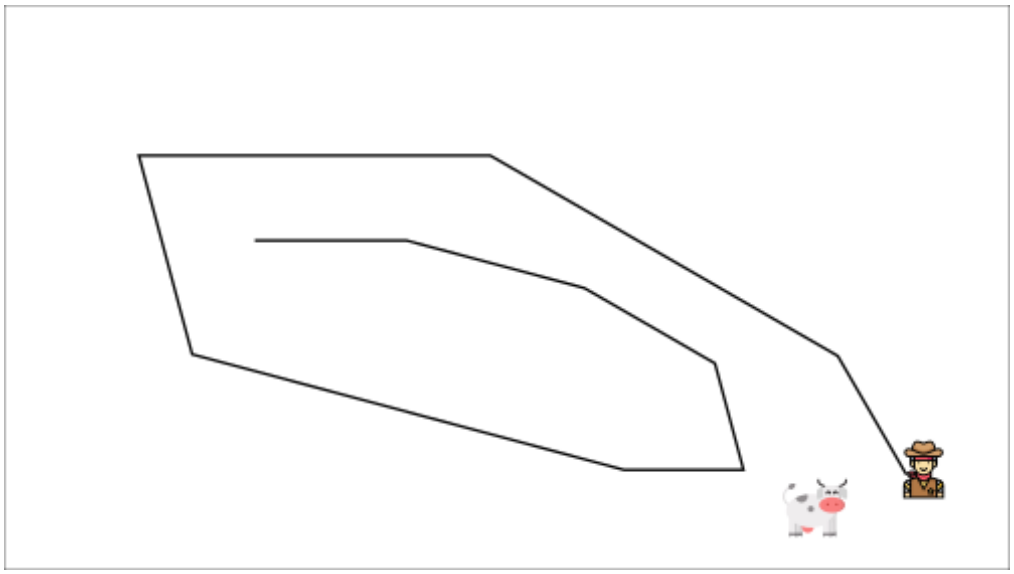


Рис. 20: Движущаяся цель: колебания при обучении функции ценности [оригинал]

Вместо этого, как видно из псевдокода, мы:

- Используем **отдельную сеть с фиксированными параметрами** для оценки целевого TD-значения.
- **Копируем параметры основной глубокой Q-сети каждые C шагов**, обновляя целевую сеть.

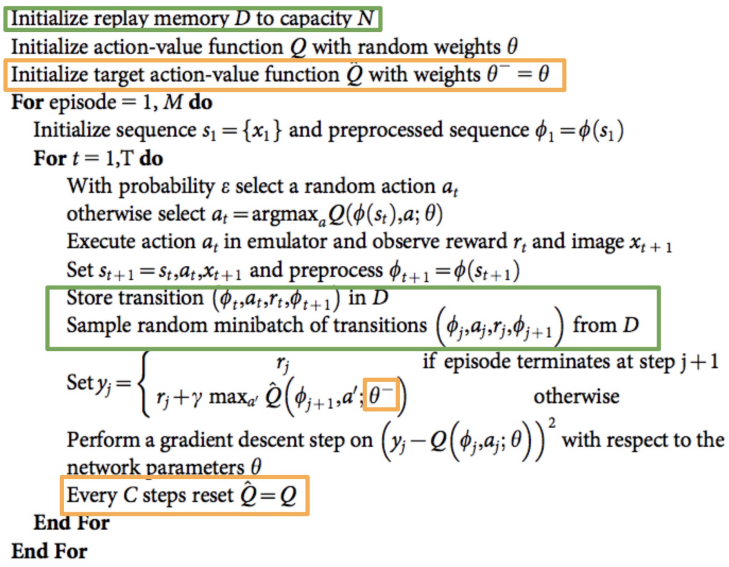


Рис. 21: Псевдокод DQN: обновление фиксированной целевой сети [оригинал]

4.3 Двойная DQN (Double DQN)

Алгоритм Double DQN, или двойное глубокое Q-обучение, предложил Хадо ван Хасселт. Этот метод **решает проблему переоценки Q-значений**.

Чтобы понять проблему, вспомним, как вычисляется целевое TD-значение:

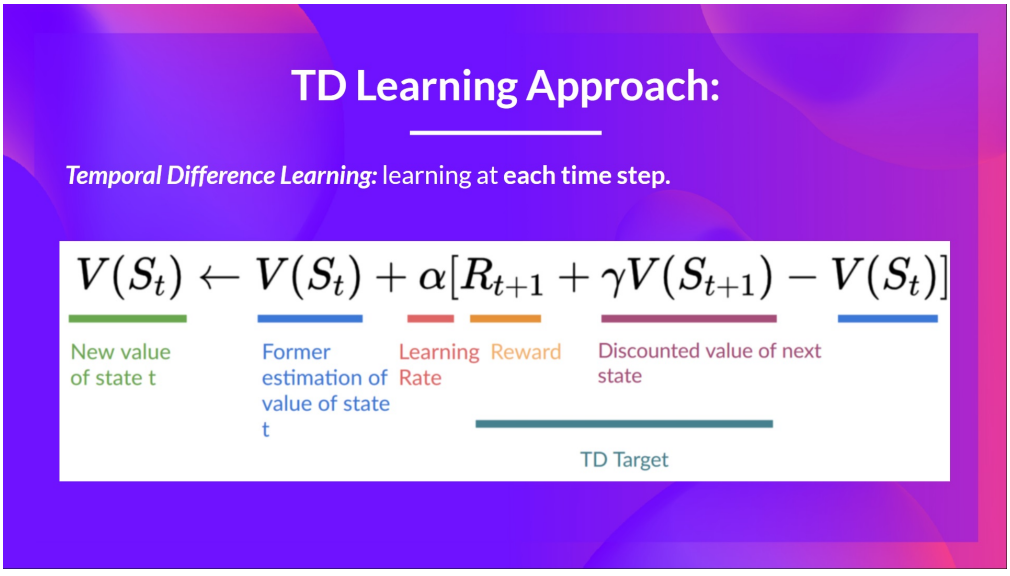


Рис. 22: Цель TD и операция максимизации по Q-значениям [оригинал]

При вычислении целевого TD-значения возникает простой вопрос: откуда мы знаем, что **лучшее действие для следующего состояния — это именно действие с максимальным Q-значением**?

Нам известно, что точность Q-оценок зависит и от того, **какие действия мы уже пробовали**, и от того, какие соседние состояния исследовали.

Поэтому в начале обучения информации о действительно лучшем действии недостаточно. Если считать максимум зашумлённых Q-оценок лучшим действием, можно получить ложноположительные результаты. Когда неоптимальным действиям регулярно **приписываются более высокие Q-значения, чем действительно оптимальному действию**, обучение осложняется.

Решение: при вычислении целевого Q-значения использовать две сети, чтобы разделить выбор

действия и получение целевой оценки ценности. Для этого мы:

- Используем **основную сеть DQN**, чтобы выбрать лучшее действие для следующего состояния (действие с наибольшим Q-значением).
- Используем **целевую сеть**, чтобы рассчитать целевое Q-значение выбранного действия в следующем состоянии.

Таким образом, Double DQN помогает уменьшить переоценку Q-значений и, как следствие, обеспечивает более быстрое и устойчивое обучение.

Помимо трёх рассмотренных усовершенствований Deep Q-Learning существуют и другие, например Prioritized Experience Replay и Dueling Deep Q-Learning. Они выходят за рамки этого курса, но при желании вы можете изучить материалы по ссылкам из списка дополнительной литературы.

5 Практическое занятие

Открыть официальный ноутбук модуля 3 в Google Colab

Официальная помощь по курсу и Discord

Теперь, когда мы изучили теорию Deep Q-Learning, **вы готовы обучить агента играть в игры Atari**. Начнём со Space Invaders, но затем сможете выбрать любую другую игру Atari.

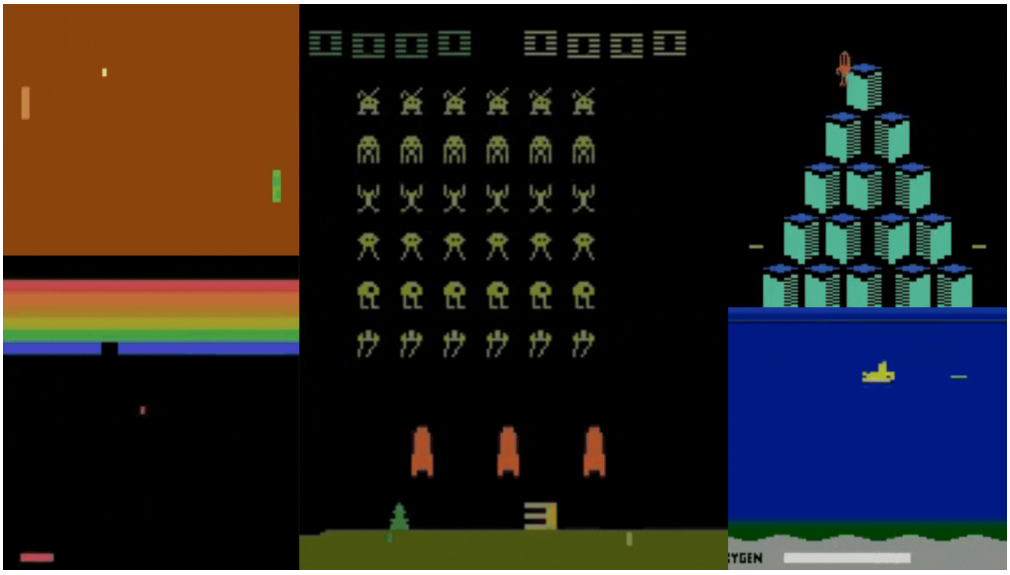


Рис. 23: Примеры сред Atari, в которых можно обучать агентов [оригинал]

Мы используем интеграцию RL-Baselines3 Zoo: обычную реализацию Deep Q-Learning без расширений Double-DQN, Dueling-DQN и Prioritized Experience Replay.

Если после практики вы захотите **реализовать Deep Q-Learning самостоятельно**, обязательно посмотрите реализацию CleanRL: https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/dqn_atari.py

Чтобы практическое задание засчиталось для сертификации, необходимо загрузить обученную модель в Hub и **получить результат не ниже 200**.

Чтобы узнать результат, найдите модель в таблице лидеров: **результат = mean_reward - std вознаграждения**.

Если модели нет в списке, прокрутите страницу вниз и нажмите кнопку обновления.

Подробнее о сертификации см. в соответствующем разделе курса.

А здесь можно проверить собственный прогресс.

Чтобы начать практику, нажмите кнопку «Open In Colab»:

Открыть в Colab

5.1 Модуль 3: Deep Q-Learning в играх Atari (Space Invaders) с помощью RL Baselines3 Zoo

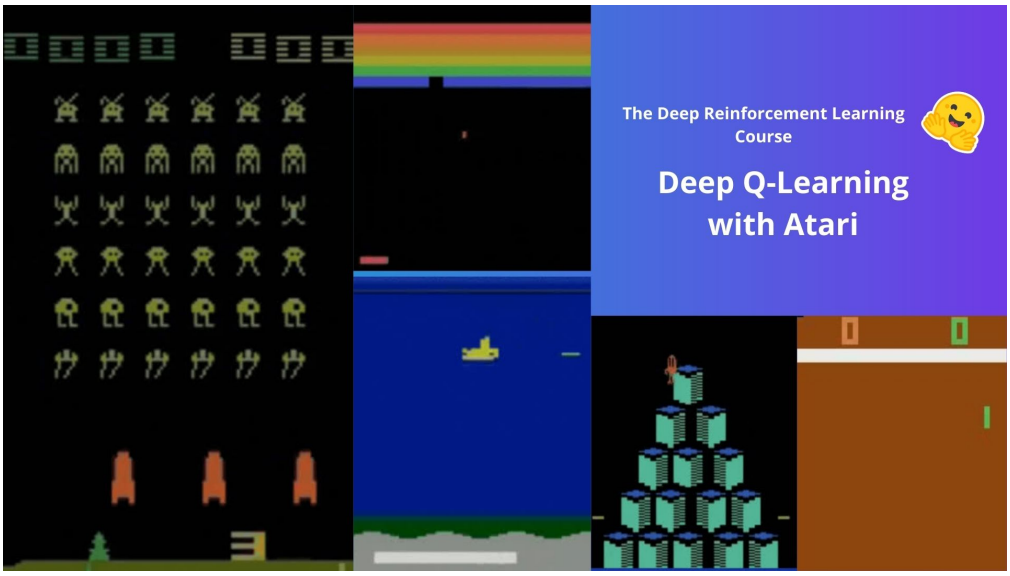


Рис. 24: Обложка модуля 3: глубокое Q-обучение [оригинал]

В этой практической работе **вы обучите агента Deep Q-Learning играть в Space Invaders** с помощью RL Baselines3 Zoo — инструментария на основе Stable-Baselines3, содержащего скрипты для обучения и оценки агентов, настройки гиперпараметров, построения графиков и записи видео.

Мы используем интеграцию RL-Baselines3 Zoo с базовой реализацией Deep Q-Learning без таких расширений, как Double-DQN, Dueling-DQN и Prioritized Experience Replay.

Игровые среды

- SpacesInvadersNoFrameskip-v4

Различия между версиями Space Invaders описаны здесь.

Библиотека RL для изучения

- RL-Baselines3-Zoo

Цели практического занятия

После выполнения работы вы сможете:

- Глубже понять, **как работает RL Baselines3 Zoo**.
- **Загрузить обученного агента и код в Hugging Face Hub** вместе с красивой видеозаписью игры и результатами оценки.

Предварительные требования

Перед выполнением практического задания необходимо:

[Чтение] **Изучить теорию Deep Q-Learning из модуля 3**.

Мы постоянно улучшаем наши руководства. Поэтому, **если вы заметили проблему в практическом задании**, пожалуйста, создайте issue в репозитории GitHub.

5.2 Обучим агента Deep Q-Learning играть в Atari Space Invaders и загрузим его в Hub

Мы настоятельно рекомендуем студентам **выполнять практические задания в Google Colab, а не на личных компьютерах**.

Используя Google Colab, **можно сосредоточиться на обучении и экспериментах, не отвлекаясь на технические вопросы настройки окружения**.

Чтобы практическое задание засчиталось для сертификации, нужно загрузить обученную модель в Hub и **получить результат не ниже 200**.

Чтобы найти результат, откройте таблицу лидеров и найдите модель. **Результат вычисляется как mean_reward минус стандартное отклонение вознаграждения**.

Подробнее о сертификации см. в разделе курса.

Настройка GPU

- Чтобы **ускорить обучение агента, воспользуемся GPU**. В Colab выберите `Runtime > Change Runtime type`.

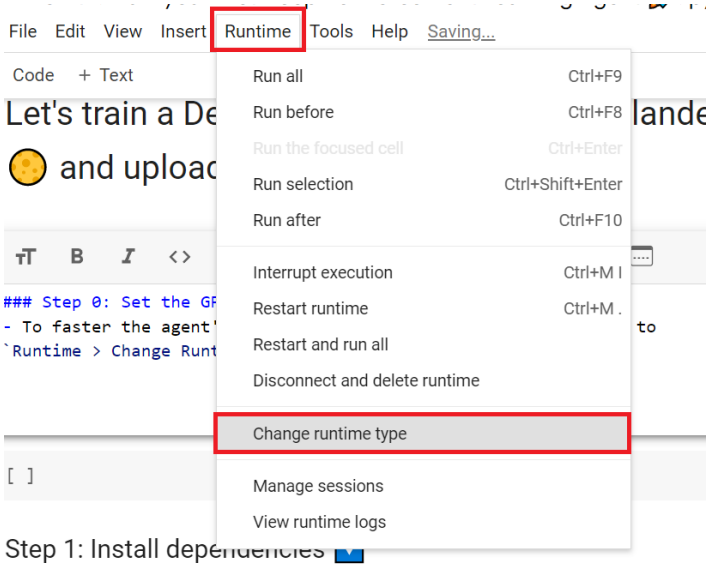


Рис. 25: Google Colab: изменение типа среды выполнения [оригинал]

- `Hardware Accelerator > GPU`

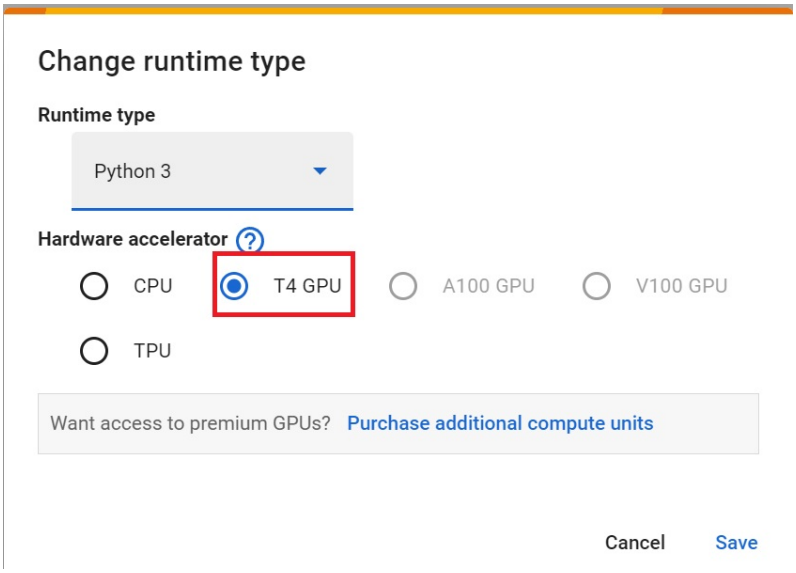


Рис. 26: Google Colab: выбор ускорителя GPU [оригинал]

5.3 Установка RL-Baselines3 Zoo и зависимостей (материал для чтения)

Если появится сообщение `ERROR: pip's dependency resolver does not currently take into account all the packages that are installed.`, **это нормально и не является критической ошибкой**: возник конфликт версий, но необходимые нам пакеты установлены.

```
# For now we install this update of RL-Baselines3 Zoo
pip install git+https://github.com/DLR-RM/rl-baselines3-zoo
```

```
apt-get install swig cmake ffmpeg
```

Для игр Atari в Gymnasium необходимо установить пакет поддержки Atari. Кроме того, нужно принять лицензию ROM-файлов с играми с помощью пакета `accept-rom-license`.

```
!pip install gymnasium[atari]
!pip install gymnasium[accept-rom-license]
```

Создание виртуального дисплея

Во время практики понадобится записать видео игры. Если обучение выполняется на компьютере без монитора (headless), **необходимо создать виртуальный экран, чтобы среда могла отрисовывать кадры** и их можно было записать.

Следующая ячейка устанавливает библиотеки, создаёт виртуальный экран и запускает его.

```
apt install python-opengl
apt install ffmpeg
apt install xvfb
pip3 install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Обучение агента Deep Q-Learning игре Space Invaders

Чтобы обучить агента с помощью RL-Baselines3-Zoo, достаточно сделать две вещи:

1. Создать файл конфигурации гиперпараметров обучения с именем `dqn.yml`.

Вот пример шаблона:

```
SpaceInvadersNoFrameskip-v4:
  env_wrapper:
    - stable_baselines3.common.atari_wrappers.AtariWrapper
  frame_stack: 4
  policy: 'CnnPolicy'
  n_timesteps: !!float 1e7
  buffer_size: 100000
  learning_rate: !!float 1e-4
  batch_size: 32
  learning_starts: 100000
  target_update_interval: 1000
  train_freq: 4
  gradient_steps: 1
```

```
  exploration_fraction: 0.1
  exploration_final_eps: 0.01
  # If True, you need to deactivate handle_timeout_termination
  # in the replay_buffer_kwargs
  optimize_memory_usage: False
```

Здесь можно увидеть следующее:

- Мы используем `Atari Wrapper`, который предобрабатывает вход (уменьшает изображение, переводит в оттенки серого и объединяет четыре кадра).
- Мы используем `CnnPolicy`, поскольку обработку кадров выполняют свёрточные слои.
- Обучаем агента на протяжении 10 миллионов временных шагов (`n_timesteps`).
- Размер буфера Experience Replay составляет 100000 — столько переходов сохраняется для последующего повторного обучения агента.

Совет: рекомендую **уменьшить число шагов обучения до одного миллиона**. На GPU P100 это займёт около 90 минут. Команда `!nvidia-smi` покажет, какой GPU у вас используется. На 10 миллионов шагов потребуется около девяти часов. Такое обучение лучше выполнять на собственном компьютере или другой машине. Для скачивания ноутбука выберите `File>Download`.

При оптимизации гиперпараметров рекомендую сосредоточиться на трёх параметрах:

- `learning_rate`
- `buffer_size` (размер памяти опыта)
- `batch_size`

Полезная привычка — **сверяться с документацией и понимать назначение каждого гиперпараметра**: https://stable-baselines3.readthedocs.io/en/master/modules/dqn.html#parameters

2. Начинаем обучение и сохраняем модели в папку `logs`.

- Указываем алгоритм после `--algo`, папку сохранения модели после `-f`, а файл конфигурации гиперпараметров после `-c`.

```
python -m rl_zoo3.train --algo ________ --env SpaceInvadersNoFrameskip-v4  -f _________  -c
    _________
```

Решение

```
python -m rl_zoo3.train --algo dqn  --env SpaceInvadersNoFrameskip-v4 -f logs/ -c dqn.yml
```

Оценим нашего агента

- RL-Baselines3-Zoo предоставляет `enjoy.py` — Python-скрипт для оценки агента. Во многих библиотеках RL скрипт оценки называется именно `enjoy.py`.
- Оценим его на протяжении 5000 временных шагов.

```
python -m rl_zoo3.enjoy  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --no-render  --n-timesteps
    _________  --folder logs/
```

Решение

```
python -m rl_zoo3.enjoy  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --no-render  --n-timesteps
    5000  --folder logs/
```

Публикация обученной модели в Hub

Убедившись, что после обучения получены хорошие результаты, мы можем опубликовать обученную модель в Hub одной строкой кода.

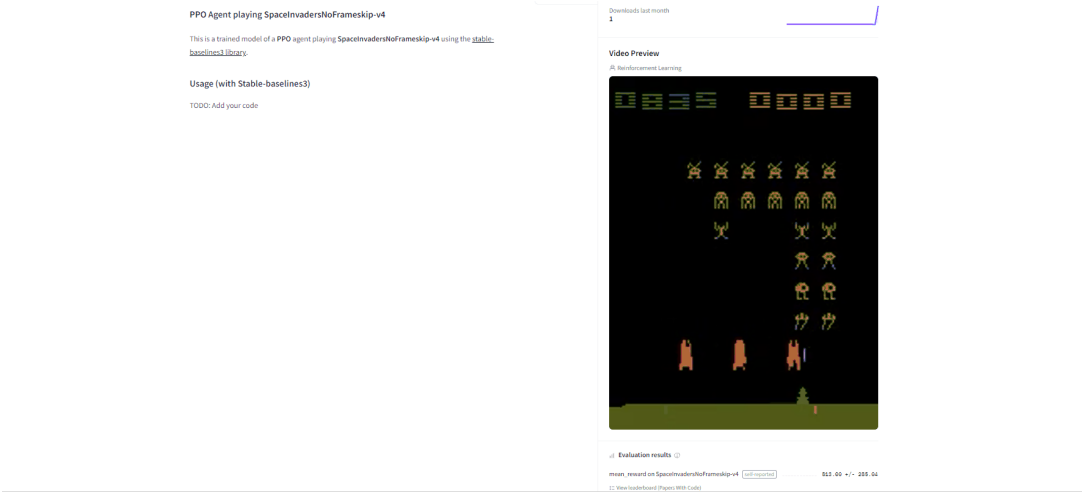


Рис. 27: Демонстрация обученного агента Space Invaders (первый кадр) [оригинал]

Команда `rl_zoo3.push_to_hub` **проводит оценку, записывает видео, создаёт карточку модели и загружает агента в Hub**.

Таким образом, вы сможете:

- **Показать результаты своей работы**.
- **Посмотреть на игру обученного агента**.
- **Поделиться с сообществом агентом, которым смогут пользоваться другие**.
- **Сравнить результаты агента с результатами других участников в таблице лидеров**: https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard

Для публикации модели в сообществе осталось выполнить ещё три шага:

1. Если аккаунта ещё нет, создайте его на Hugging Face: https://huggingface.co/join
2. Войдите в аккаунт, затем сохраните токен аутентификации, полученный на сайте Hugging Face.

- Создайте новый токен (https://huggingface.co/settings/tokens) **с правом записи (write)**.

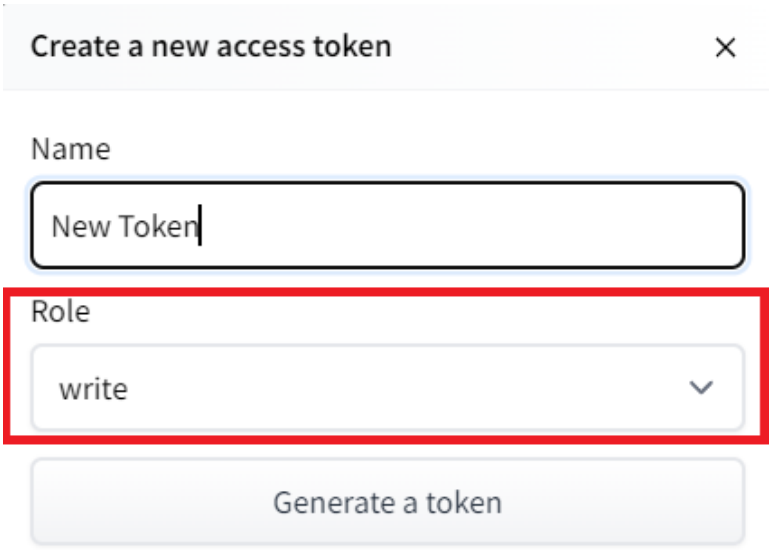


Рис. 28: Создание токена доступа Hugging Face [оригинал]

- Скопируйте токен.
- Запустите следующую ячейку и вставьте токен.

```
from huggingface_hub import notebook_login # To log to our Hugging Face account to be able to
    upload models to the Hub.
notebook_login()
!git config --global credential.helper store
```

Если вы не используете Google Colab или Jupyter Notebook, вместо этого необходимо выполнить команду `huggingface-cli login`.

3. Теперь всё готово для отправки обученного агента в Hub.

Запустим файл `push_to_hub.py`, чтобы загрузить обученного агента в Hub.

`--repo-name`: название репозитория.

`-orga`: ваше имя пользователя на Hugging Face.

`-f`: папка, в которой находится обученная модель (в нашем случае `logs`).

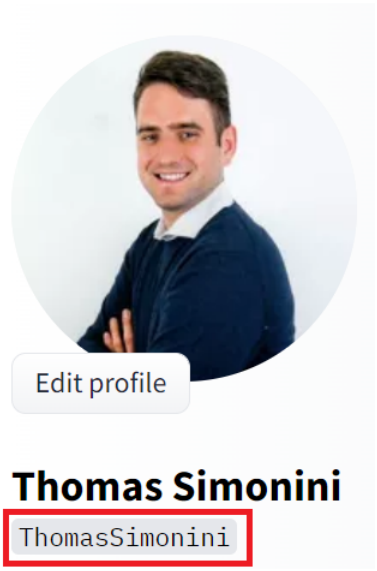


Рис. 29: Выбор каталога с обученной моделью для публикации [оригинал]

```
python -m rl_zoo3.push_to_hub  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --repo-name
    ____________________ -orga ____________________ -f logs/
```

Решение

```
python -m rl_zoo3.push_to_hub  --algo dqn  --env SpaceInvadersNoFrameskip-v4  --repo-name dqn-
    SpaceInvadersNoFrameskip-v4  -orga ThomasSimonini  -f logs/
```

Поздравляем: вы только что обучили и загрузили своего первого агента Deep Q-Learning с помощью RL-Baselines3 Zoo! Скрипт выше должен был вывести ссылку на репозиторий модели, например https://huggingface.co/ThomasSimonini/dqn-SpaceInvadersNoFrameskip-v4. Перейдя по ссылке, вы сможете:

- Увидеть **предпросмотр видео с игрой агента** справа.
- Нажать «Files and versions» и просмотреть все файлы репозитория.
- Нажать «Use in stable-baselines3», чтобы получить пример кода для загрузки модели.
- Просмотреть карточку модели (файл `README.md`) с её описанием и гиперпараметрами обучения.

В основе Hub лежат репозитории Git (не беспокойтесь, если вы ещё не знакомы с Git). Поэтому можно публиковать новые версии модели по мере проведения экспериментов и улучшения агента.

Сравните результаты своих агентов с результатами других участников в таблице лидеров.

Загрузка хорошо обученной модели

- Команда Stable-Baselines3 опубликовала в Hub **более 150 обученных агентов глубокого обучения с подкреплением**.

Найти их можно здесь: https://huggingface.co/sb3

Несколько примеров:

- Asteroids: https://huggingface.co/sb3/dqn-AsteroidsNoFrameskip-v4
- Beam Rider: https://huggingface.co/sb3/dqn-BeamRiderNoFrameskip-v4
- Breakout: https://huggingface.co/sb3/dqn-BreakoutNoFrameskip-v4
- Road Runner: https://huggingface.co/sb3/dqn-RoadRunnerNoFrameskip-v4

Загрузим агента, играющего в Beam Rider. Его модель размещена здесь: https://huggingface.co/sb3/dqn-BeamRiderNoFrameskip-v4

1. Скачаем модель с помощью `rl_zoo3.load_from_hub` и поместим в новую папку, которую назовём `rl_trained`.

```
# Download model and save it into the logs/ folder
python -m rl_zoo3.load_from_hub --algo dqn --env BeamRiderNoFrameskip-v4 -orga sb3 -f rl_trained/
```

2. Оценим модель на протяжении 5000 временных шагов.

```
python -m rl_zoo3.enjoy --algo dqn --env BeamRiderNoFrameskip-v4 -n 5000  -f rl_trained/ --no-render
```

Почему бы не попробовать самостоятельно обучить **агента Deep Q-Learning** для игры Beam Rider (`BeamRiderNoFrameskip-v4`)?

Если захотите попробовать, изучите карточку обученной модели: https://huggingface.co/sb3/dqn-BeamRiderNoFrameskip-v4#hyperparameters

В ней приведены использованные гиперпараметры обучения.

Однако подбор гиперпараметров бывает непростой задачей. К счастью, в следующем модуле мы увидим, как **использовать Optuna для оптимизации гиперпараметров**.

Дополнительные задания

Лучший способ учиться — **пробовать всё самостоятельно**!

В таблице лидеров вы найдёте своих агентов. Сможете ли вы подняться на первую строчку?

Вот несколько сред, в которых можно попробовать обучить агента:

- BeamRiderNoFrameskip-v4
- BreakoutNoFrameskip-v4
- EnduroNoFrameskip-v4
- PongNoFrameskip-v4

Если вы хотите **самостоятельно реализовать Deep Q-Learning**, обязательно ознакомьтесь с реализацией CleanRL: https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/dqn_atari.py

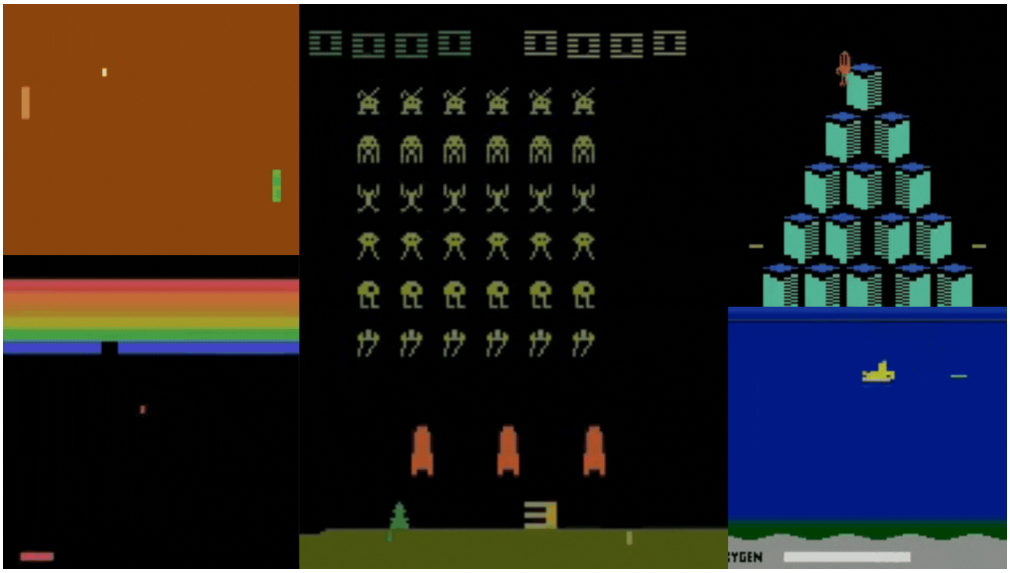


Рис. 30: Примеры сред Atari, в которых можно обучать агентов [оригинал]

Поздравляем с завершением этой главы!

Если некоторые понятия до сих пор остаются непонятными… это совершенно нормально! **Так было и у меня, и у всех, кто изучал обучение с подкреплением**.

Прежде чем двигаться дальше, уделите время тому, чтобы **по-настоящему разобраться в материале и попробовать дополнительные задания**. Важно хорошо освоить эти идеи и сформировать прочную основу.

В следующем модуле **мы познакомимся с Optuna**. Одна из самых важных задач глубокого обучения с подкреплением — подобрать хорошие гиперпараметры обучения. Библиотека Optuna помогает автоматизировать их поиск.

Этот курс создаётся вместе с вами Мы хотим постоянно улучшать и обновлять курс с учётом ваших отзывов. Если есть предложения, заполните, пожалуйста, эту форму.

Мы постоянно улучшаем наши руководства, поэтому **если вы заметили ошибку в ноутбуке**, создайте issue в GitHub-репозитории.

Увидимся во втором бонусном модуле!

Продолжайте учиться, оставайтесь замечательными!

6 Проверка знаний

Лучший способ учиться и избегать иллюзии компетентности — **проверять себя**. Так вы поймёте, **какие темы необходимо повторить**.

Вопрос 1. Мы называли Q-обучение табличным методом. Что такое табличные методы?

Вопрос 2. Почему классическое Q-обучение не подходит для решения игр Atari?

- **A.** Среды Atari слишком быстрые для Q-обучения.
- **B.** В средах Atari огромное пространство наблюдений, поэтому создавать и обновлять Q-таблицу было бы неэффективно.

Вопрос 3. Почему при использовании изображений в Deep Q-Learning мы объединяем четыре последовательных кадра?

Вопрос 4. Какие две фазы есть в Deep Q-Learning?

- **A.** Сбор опыта (sampling).
- **B.** Перемешивание (shuffling).
- **C.** Повторное ранжирование (reranking).
- **D.** Обучение (training).

Вопрос 5. Зачем в Deep Q-Learning нужна память воспроизведения опыта?

Вопрос 6. Как работает Double Deep Q-Learning?

Поздравляем с завершением теста! Если какие-то вопросы оказались сложными, перечитайте соответствующие разделы, чтобы закрепить знания.

6.1 Ответы на вопросы и пояснения

Ответ на вопрос 1

Табличные методы применимы к задачам, в которых пространства состояний и действий достаточно малы, чтобы аппроксимированные функции ценности можно было **представить массивами и таблицами**. Например, **Q-обучение — табличный метод**, поскольку для хранения ценностей пар «состояние–действие» используется таблица.

Ответ на вопрос 2

Правильный ответ: B.

A. Среды Atari слишком быстрые для Q-обучения.

B. В средах Atari огромное пространство наблюдений, поэтому создавать и обновлять Q-таблицу было бы неэффективно.

Ответ на вопрос 3

Мы объединяем кадры, чтобы **решить проблему нехватки временной информации**: по одному кадру невозможно определить движение. Например, в Pong агент **не сможет понять направление движения мяча, если получит только один кадр**.

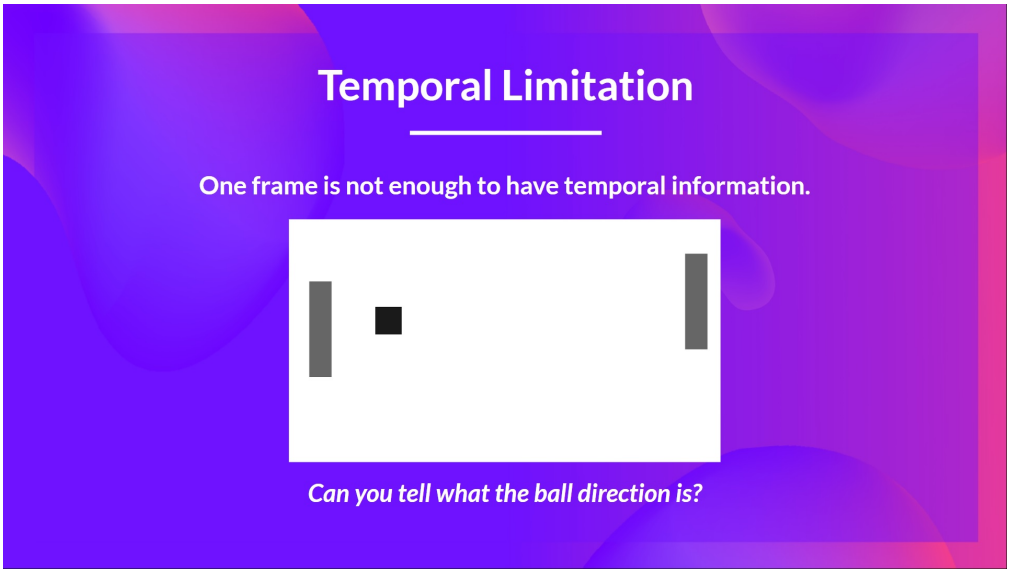


Рис. 31: По одному кадру Pong нельзя определить скорость и направление движения мяча [оригинал]

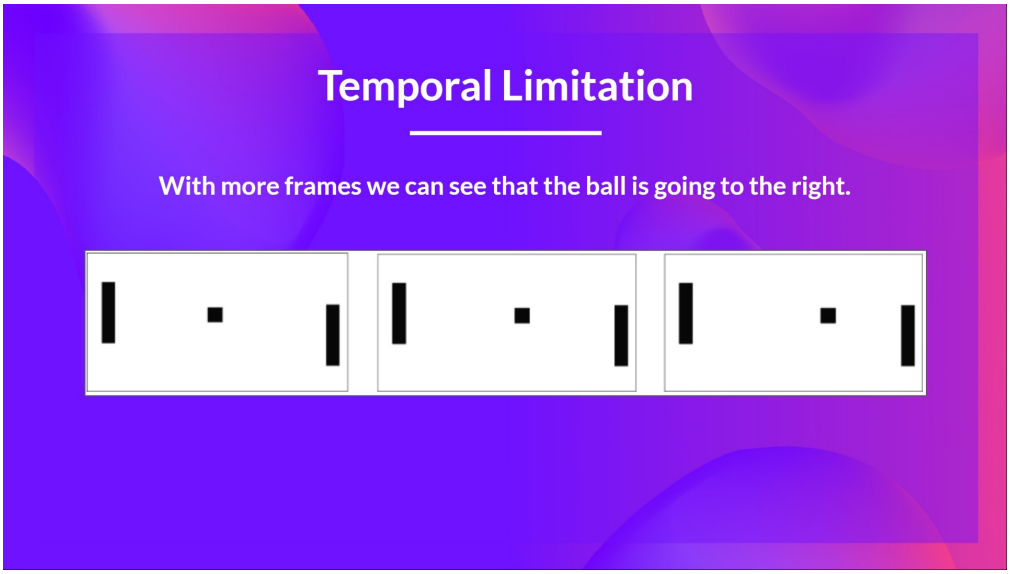


Рис. 32: Несколько последовательных кадров Pong позволяют определить направление мяча [оригинал]

Ответ на вопрос 4

Правильные ответы: A и D.

A. Сбор опыта. Пояснение: агент совершает действия и сохраняет наблюдаемые переходы в памяти воспроизведения.

B. Перемешивание.

C. Повторное ранжирование.

D. Обучение. Пояснение: случайным образом выбираем небольшой пакет переходов и обучаемся на нём, обновляя веса градиентным спуском.

Ответ на вопрос 5

1. Более эффективное использование опыта во время обучения.

При обычном онлайн-обучении с подкреплением агент взаимодействует со средой, получает переходы (состояние, действие, вознаграждение и следующее состояние), учится на них, обновляя нейронную сеть, а затем отбрасывает. Это неэффективно. При Experience Replay **создаётся буфер, в котором хранятся переходы, пригодные для повторного использования во время обучения**.

2. Предотвращение забывания и уменьшение корреляции между переходами.

При последовательной подаче переходов нейронная сеть **склонна забывать прежний опыт по мере запоминания нового**. Например, если после первого уровня агент попадает на сильно отличающийся второй, он может забыть, как правильно играть на первом уровне.

Ответ на вопрос 6

При вычислении целевого Q-значения мы используем две сети, чтобы разделить выбор действия и формирование целевого значения. Для этого мы:

- Используем *основную сеть DQN*, чтобы **выбрать лучшее действие для следующего состояния** (действие с максимальным Q-значением).
- Используем *целевую сеть*, чтобы **рассчитать целевое Q-значение выбранного действия в следующем состоянии**.

7 Заключение

Поздравляем с завершением главы! В ней было много информации. Поздравляем также с завершением практического занятия: вы только что обучили своего первого агента Deep Q-Learning и опубликовали его в Hub.

Прежде чем двигаться дальше, выделите время на то, чтобы хорошо усвоить материал.

Не бойтесь обучать агента в других средах (Pong, Seaquest, QBert, Ms Pac Man). **Лучший способ научиться — пробовать всё самостоятельно!**

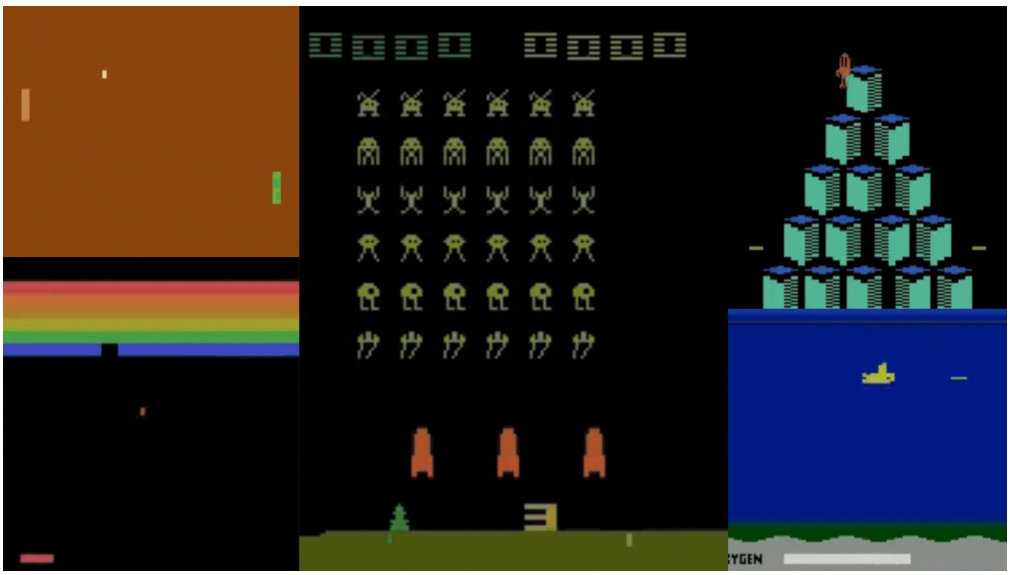


Рис. 33: Примеры сред Atari, в которых можно обучать агентов [оригинал]

В следующем модуле **мы познакомимся с Optuna**. Одна из важнейших задач глубокого обучения с подкреплением — подбор хорошего набора гиперпараметров. Библиотека Optuna помогает автоматизировать их поиск.

Нам также очень важно **услышать, что вы думаете о курсе и как его можно улучшить**. Если есть отзывы, пожалуйста, заполните эту форму.

Продолжайте учиться и оставайтесь замечательными!

8 Глоссарий

Этот глоссарий создан сообществом. Мы рады новым предложениям и дополнениям!

- **Табличный метод (Tabular Method):** подход для задач с достаточно небольшими пространствами состояний и действий, позволяющими представлять аппроксимированные функции ценности массивами и таблицами. Например, **Q-обучение** — табличный метод, так как для хранения ценности различных пар «состояние–действие» используется таблица.
- **Глубокое Q-обучение (Deep Q-Learning):** метод, обучающий нейронную сеть аппроксимировать **Q-значения** всех возможных действий по текущему состоянию. Он применяется, когда пространство наблюдений слишком велико для табличного Q-обучения.
- **Проблема временной информации (Temporal Limitation):** трудность, возникающая, когда состояние среды представлено отдельными изображениями. Один кадр не передаёт информацию о движении. Чтобы получить её, необходимо **объединить в стек** несколько последовательных кадров.
- **Фазы Deep Q-Learning:**
  - **Сбор опыта (Sampling):** агент совершает действия, а полученные переходы сохраняются в **буфере воспроизведения опыта**.
  - **Обучение (Training):** случайным образом выбираются пакеты переходов, и нейросеть обновляет веса с помощью градиентного спуска.
- **Способы стабилизации Deep Q-Learning:**
  - **Воспроизведение опыта (Experience Replay):** создаётся память переходов, которые можно повторно использовать во время обучения. Благодаря этому агент многократно учится на одном и том же опыте. Кроме того, это помогает не забывать прежние переходы по мере поступления новых.
  - **Случайная выборка** из буфера уменьшает корреляцию между последовательными наблюдениями и предотвращает сильные колебания или расходимость ценностей действий.
  - **Фиксированная целевая Q-сеть (Fixed Q-Target):** для вычисления **целевого Q-значения** с помощью уравнения Беллмана необходимо оценить дисконтированную оптимальную **Q-ценность** следующего состояния. Проблема в том, что для вычисления **Q-Target** и текущей **Q-оценки** используются одни и те же веса сети. Поэтому при каждом изменении **Q-оценки** смещается и **цель**. Чтобы избежать этого, для оценки целевого значения временных разностей применяется отдельная сеть с фиксированными параметрами. Её обновляют, копируя веса основной DQN через определённое число **шагов** C.
  - **Double DQN:** метод борьбы с **переоценкой Q-значений**. В нём применяются две сети, позволяющие разделить выбор действия и вычисление целевого **значения ценности**:
    * **Основная сеть DQN** выбирает лучшее действие в следующем состоянии (с максимальным **Q-значением**).
    * **Целевая сеть** вычисляет целевое **Q-значение** выбранного действия в следующем состоянии. Такой подход уменьшает переоценку Q-значений, ускоряет обучение и делает его более устойчивым.

Если вы хотите улучшить курс, можете создать Pull Request.

Создание этого глоссария стало возможным благодаря:

- Дарио Паэсу (Dario Paez).

9 Дополнительная литература

Эти материалы **необязательны**, но помогут глубже разобраться в теме.

- Серия Foundations of Deep RL, лекция 2 «Deep Q-Learning», Питер Аббил
- Игра в Atari с глубоким обучением с подкреплением (Playing Atari with Deep Reinforcement Learning)
- Двойное глубокое Q-обучение (Double Deep Q-Learning)
- Приоритизированное воспроизведение опыта (Prioritized Experience Replay)
- Дуэльное глубокое Q-обучение (Dueling Deep Q-Learning)